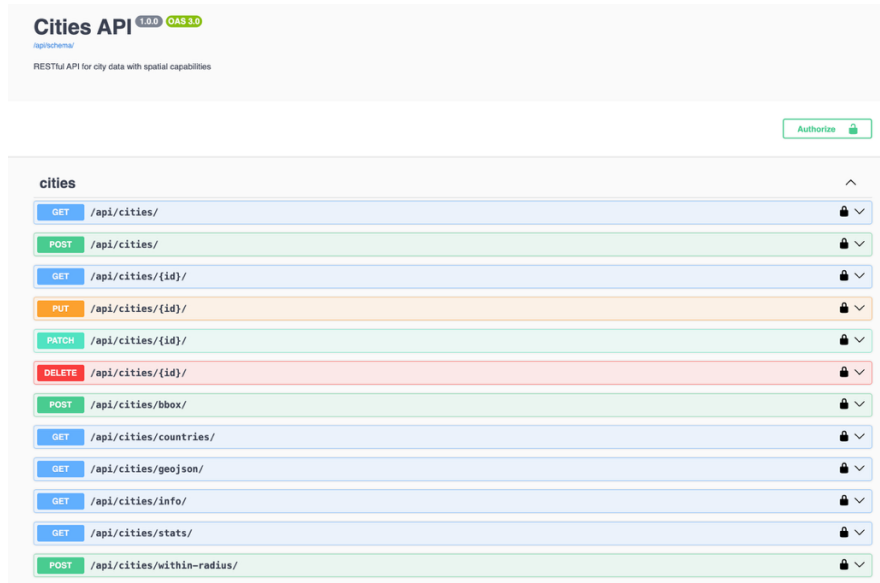


LAB 3

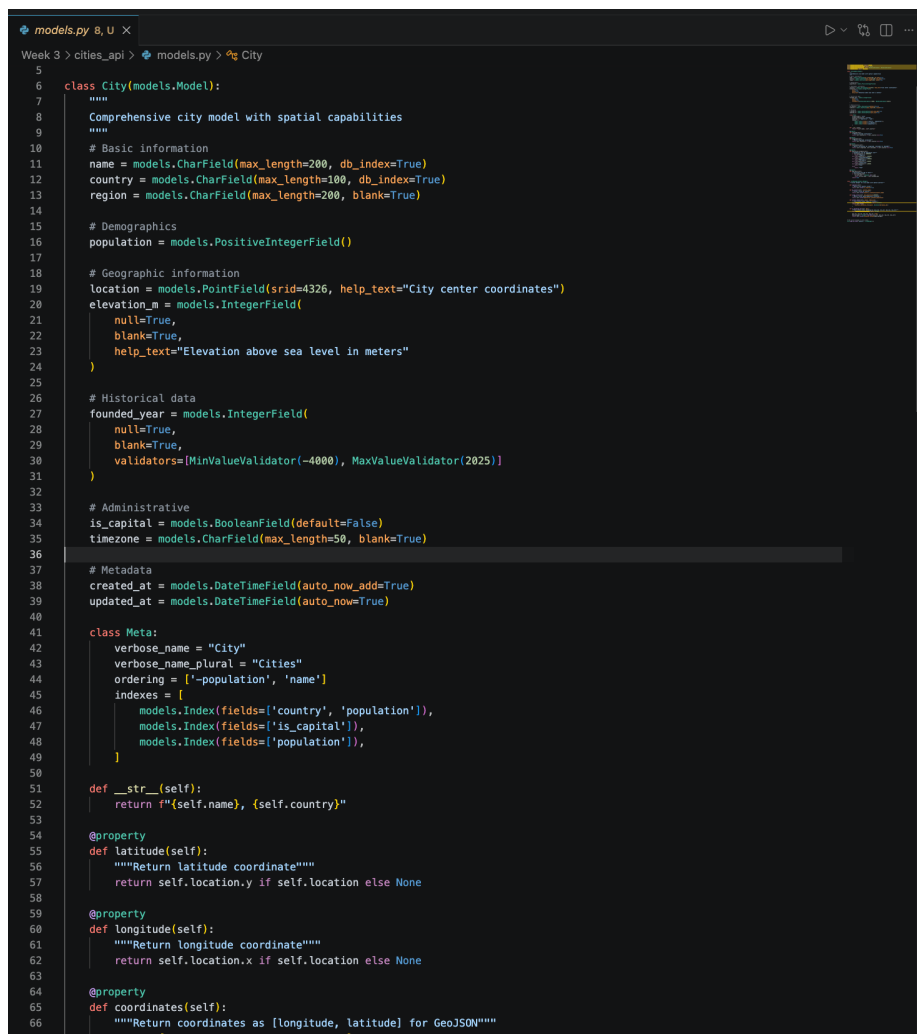
Required Submissions

1. Complete Django Project:

- Working API app with all endpoints: <http://127.0.0.1:8000/api/docs/>



- Comprehensive city model with spatial fields



- API serialisers for different output formats

```
models.py 8, U serializers.py 3, U X
Week 3 > cities_api > serializers.py > BoundingBoxSerializer > validate
5 class CityListSerializer(serializers.ModelSerializer):
6     """Lightweight serializer for city lists"""
7
8     latitude = serializers.ReadOnlyField()
9     longitude = serializers.ReadOnlyField()
10    population_category = serializers.ReadOnlyField()
11
12    class Meta:
13        model = City
14        fields = [
15            'id', 'name', 'country', 'population',
16            'latitude', 'longitude', 'is_capital',
17            'population_category'
18        ]
19
20    class CityDetailSerializer(serializers.ModelSerializer):
21        """Comprehensive serializer for city details"""
22
23        latitude = serializers.ReadOnlyField()
24        longitude = serializers.ReadOnlyField()
25        coordinates = serializers.ReadOnlyField()
26        population_category = serializers.ReadOnlyField()
27        age_years = serializers.ReadOnlyField()
28
29        class Meta:
30            model = City
31            fields = [
32                'id', 'name', 'country', 'region', 'population',
33                'latitude', 'longitude', 'coordinates',
34                'elevation_m', 'founded_year', 'age_years',
35                'is_capital', 'timezone', 'population_category',
36                'created_at', 'updated_at'
37            ]
38
39    class CityGeoJSONSerializer(GeoFeatureModelSerializer):
40        """GeoJSON serializer for mapping applications"""
41
42        population_category = serializers.ReadOnlyField()
43        age_years = serializers.ReadOnlyField()
44
45        class Meta:
46            model = City
47            geo_field = 'location'
48            fields = [
49                'id', 'name', 'country', 'region', 'population',
50                'elevation_m', 'founded_year', 'age_years',
51                'is_capital', 'timezone', 'population_category'
52            ]
53
54    class CityCreateSerializer(serializers.ModelSerializer):
55        """Serializer for creating new cities"""
56
57        latitude = serializers.FloatField(write_only=True)
58        longitude = serializers.FloatField(write_only=True)
59
60        class Meta:
61            model = City
62            fields = [
63                'name', 'country', 'region', 'population',
64                'latitude', 'longitude', 'elevation_m',
65                'founded_year', 'is_capital', 'timezone'
66            ]
67
```

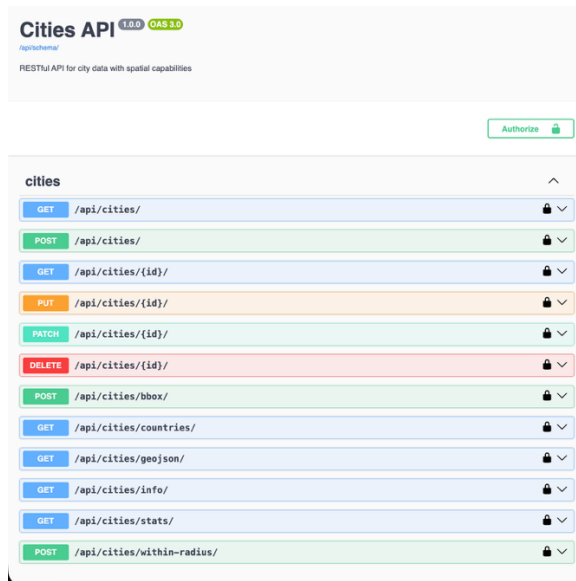
- Filtering and spatial query capabilities

```
models.py 8, U serializers.py 3, U filters.py 1, U X
Week 3 > cities_api > filters.py > CityFilter > Meta
1 import django_filters
2 from .models import City
3
4 class CityFilter(django_filters.FilterSet):
5     """Filter set for City model"""
6
7     name = django_filters.CharFilter(lookup_expr='icontains')
8     country = django_filters.CharFilter(lookup_expr='icontains')
9     region = django_filters.CharFilter(lookup_expr='icontains')
10
11     min_population = django_filters.NumberFilter(
12         field_name='population',
13         lookup_expr='gte'
14     )
15     max_population = django_filters.NumberFilter(
16         field_name='population',
17         lookup_expr='lte'
18     )
19
20     min_founded_year = django_filters.NumberFilter(
21         field_name='founded_year',
22         lookup_expr='gte'
23     )
24     max_founded_year = django_filters.NumberFilter(
25         field_name='founded_year',
26         lookup_expr='lte'
27     )
28
29     is_capital = django_filters.BooleanFilter()
30
31     class Meta:
32         model = City
33         fields = [
34             'name', 'country', 'region', 'is_capital',
35             'min_population', 'max_population',
36             'min_founded_year', 'max_founded_year'
37         ]

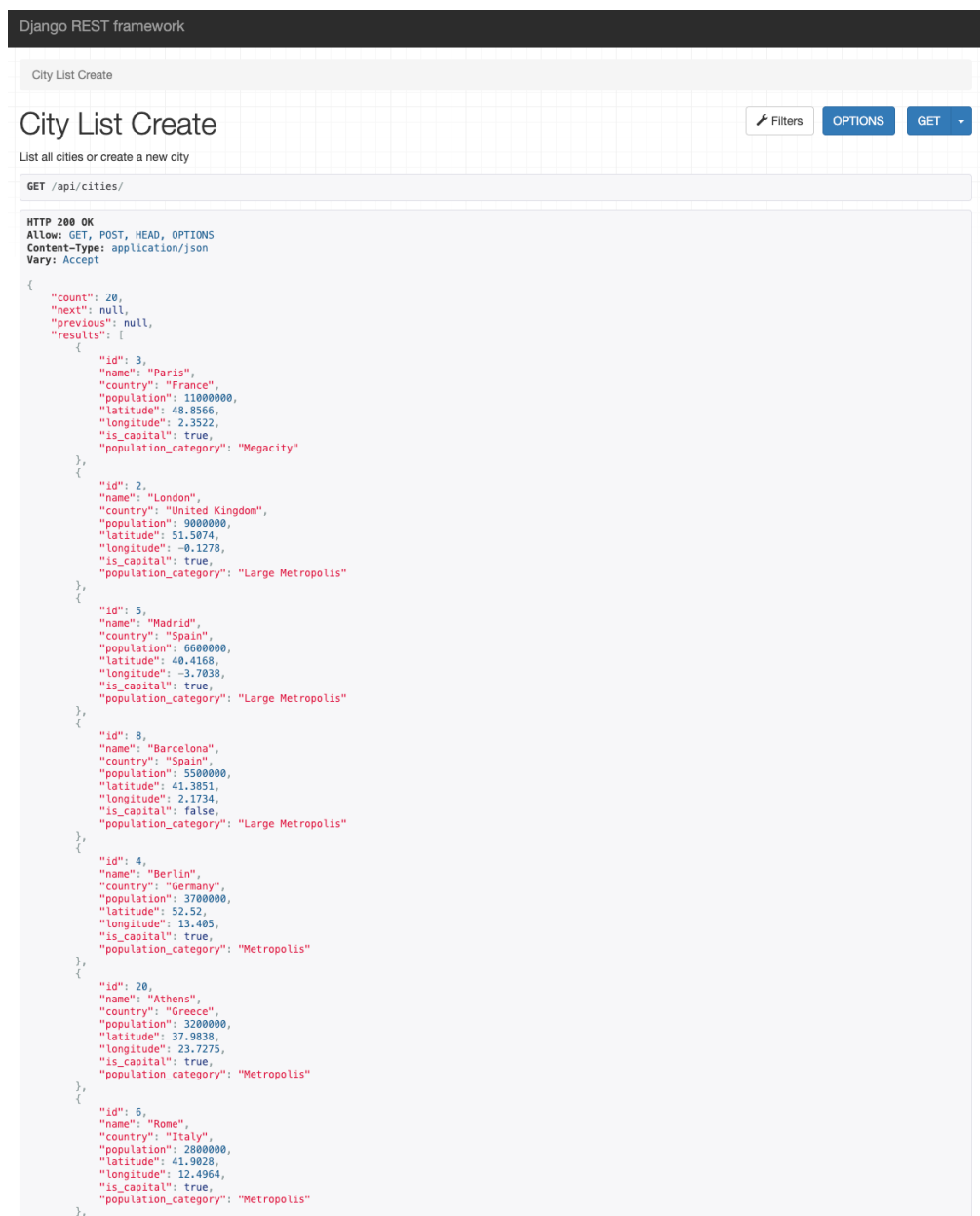
```

2. API Documentation:

- Swagger/OpenAPI documentation



- Manual testing results



```

{
  "id": 19,
  "name": "Lisbon",
  "country": "Portugal",
  "population": 2800000,
  "latitude": 38.7223,
  "longitude": -9.1393,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 10,
  "name": "Vienna",
  "country": "Austria",
  "population": 1900000,
  "latitude": 48.2052,
  "longitude": 16.3738,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 15,
  "name": "Warsaw",
  "country": "Poland",
  "population": 1800000,
  "latitude": 52.2297,
  "longitude": 21.0122,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 11,
  "name": "Stockholm",
  "country": "Sweden",
  "population": 1600000,
  "latitude": 59.3293,
  "longitude": 18.0686,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 9,
  "name": "Munich",
  "country": "Germany",
  "population": 1500000,
  "latitude": 48.1351,
  "longitude": 11.582,
  "is_capital": false,
  "population_category": "Metropolis"
},
{
  "id": 1,
  "name": "Dublin",
  "country": "Ireland",
  "population": 1400000,
  "latitude": 53.3498,
  "longitude": -6.2603,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 16,
  "name": "Prague",
  "country": "Czech Republic",
  "population": 1300000,
  "latitude": 50.0755,
  "longitude": 14.4378,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 17,
  "name": "Brussels",
  "country": "Belgium",
  "population": 1200000,
  "latitude": 50.8503,
  "longitude": 4.3517,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 7,
  "name": "Amsterdam",
  "country": "Netherlands",
  "population": 900000,
  "latitude": 52.3676,
  "longitude": 4.9041,
  "is_capital": true,
  "population_category": "Large City"
},
{
  "id": 12,
  "name": "Copenhagen",
  "country": "Denmark",
  "population": 800000,
  "latitude": 55.6761,
  "longitude": 12.5683,
  "is_capital": true,
  "population_category": "Large City"
}
}

```

```

{
  "id": 14,
  "name": "Helsinki",
  "country": "Finland",
  "population": 650000,
  "latitude": 60.1699,
  "longitude": 24.9384,
  "is_capital": true,
  "population_category": "Large City"
},
{
  "id": 18,
  "name": "Zurich",
  "country": "Switzerland",
  "population": 400000,
  "latitude": 47.3769,
  "longitude": 8.5417,
  "is_capital": false,
  "population_category": "City"
}
}

```

- API endpoint descriptions

Endpoint	Method	Purpose
/api/cities/	GET	List cities
/api/cities/	POST	Create a city
/api/cities/{id}/	GET	Retrieve a city
/api/cities/{id}/	PUT	Update a city
/api/cities/{id}/	PATCH	Partially update a city
/api/cities/{id}/	DELETE	Delete a city
/api/cities/geojson/	GET	Return cities as GeoJSON
/api/cities/within-radius/	POST	Find cities within a radius
/api/cities/bbox/	POST	Find cities within a bounding box
/api/cities/stats/	GET	Return city statistics
/api/cities/countries/	GET	Return available countries
/api/cities/info/	GET	Return API/city information

3. Testing Evidence:

- Unit tests for models and API endpoints

```

serializers.py 3, U  filters.py 1, U  test_api.py 5, U  X
Week 3 > cities_api > tests > test_api.py > CityAPITestCase > test_within_radius_query
8 class CityAPITestCase(APITestCase):
9     """Test cases for Cities API"""
10
11     def setUp(self):
12         """Create test data"""
13         self.dublin = City.objects.create(
14             name='Dublin',
15             country='Ireland',
16             population=1400000,
17             location=Point(-6.2603, 53.3498, srid=4326),
18             is_capital=True
19         )
20
21         self.london = City.objects.create(
22             name='London',
23             country='United Kingdom',
24             population=9000000,
25             location=Point(-0.1278, 51.5074, srid=4326),
26             is_capital=True
27         )
28
29     def test_city_list(self):
30         """Test city list endpoint"""
31         url = reverse('cities_api:city-list-create')
32         response = self.client.get(url)
33
34         self.assertEqual(response.status_code, status.HTTP_200_OK)
35         self.assertEqual(response.data['count'], 2)
36         self.assertIn('results', response.data)
37
38     def test_city_detail(self):
39         """Test city detail endpoint"""
40         url = reverse('cities_api:city-detail', kwargs={'pk': self.dublin.pk})
41         response = self.client.get(url)
42
43         self.assertEqual(response.status_code, status.HTTP_200_OK)
44         self.assertEqual(response.data['name'], 'Dublin')
45         self.assertEqual(response.data['country'], 'Ireland')
46
47     def test_city_creation(self):
48         """Test creating a new city"""
49         url = reverse('cities_api:city-list-create')
50         data = {
51             'name': 'Paris',
52             'country': 'France',
53             'population': 11000000,
54             'latitude': 48.8566,
55             'longitude': 2.3522,
56             'is_capital': True
57         }
58         response = self.client.post(url, data, format='json')
59
60         self.assertEqual(response.status_code, status.HTTP_201_CREATED)
61         self.assertEqual(City.objects.count(), 3)
62
63     def test_city_filtering(self):
64         """Test city filtering"""
65         url = reverse('cities_api:city-list-create')
66         response = self.client.get(url, {'country': 'Ireland'})
67
68         self.assertEqual(response.status_code, status.HTTP_200_OK)
69         self.assertEqual(response.data['count'], 1)

```

○ Manual testing screenshots

Django REST framework

City List Create

City List Create

List all cities or create a new city

GET /api/cities/

HTTP 200 OK
Allow: GET, POST, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
  "count": 20,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 3,
      "name": "Paris",
      "country": "France",
      "population": 11800000,
      "latitude": 48.8566,
      "longitude": 2.3522,
      "is_capital": true,
      "population_category": "Megacity"
    },
    {
      "id": 2,
      "name": "London",
      "country": "United Kingdom",
      "population": 9000000,
      "latitude": 51.5074,
      "longitude": -0.1278,
      "is_capital": true,
      "population_category": "Large Metropolis"
    },
    {
      "id": 5,
      "name": "Madrid",
      "country": "Spain",
      "population": 6600000,
      "latitude": 40.4168,
      "longitude": -3.7038,
      "is_capital": true,
      "population_category": "Large Metropolis"
    },
    {
      "id": 8,
      "name": "Barcelona",
      "country": "Spain",
      "population": 5500000,
      "latitude": 41.3851,
      "longitude": 2.1734,
      "is_capital": false,
      "population_category": "Large Metropolis"
    },
    {
      "id": 4,
      "name": "Berlin",
      "country": "Germany",
      "population": 3700000,
      "latitude": 52.52,
      "longitude": 13.405,
      "is_capital": true,
      "population_category": "Metropolis"
    },
    {
      "id": 20,
      "name": "Athens",
      "country": "Greece",
      "population": 3200000,
      "latitude": 37.9838,
      "longitude": 23.7275,
      "is_capital": true,
      "population_category": "Metropolis"
    },
    {
      "id": 6,
      "name": "Rome",
      "country": "Italy",
      "population": 2800000,
      "latitude": 41.9028,
      "longitude": 12.4964,
      "is_capital": true,
      "population_category": "Metropolis"
    },
    {
      "id": 19,
      "name": "Lisbon",
      "country": "Portugal",
      "population": 2800000,
      "latitude": 38.7223,
      "longitude": -9.1393,

```

```
{
  "id": 19,
  "name": "Lisbon",
  "country": "Portugal",
  "population": 2800000,
  "latitude": 38.7223,
  "longitude": -9.1393,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 10,
  "name": "Vienna",
  "country": "Austria",
  "population": 1900000,
  "latitude": 48.2082,
  "longitude": 16.3730,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 15,
  "name": "Warsaw",
  "country": "Poland",
  "population": 1800000,
  "latitude": 52.2297,
  "longitude": 21.0122,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 11,
  "name": "Stockholm",
  "country": "Sweden",
  "population": 1600000,
  "latitude": 59.3293,
  "longitude": 18.0686,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 9,
  "name": "Munich",
  "country": "Germany",
  "population": 1500000,
  "latitude": 48.1351,
  "longitude": 11.582,
  "is_capital": false,
  "population_category": "Metropolis"
},
{
  "id": 1,
  "name": "Dublin",
  "country": "Ireland",
  "population": 1400000,
  "latitude": 53.3498,
  "longitude": -6.2603,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 16,
  "name": "Prague",
  "country": "Czech Republic",
  "population": 1300000,
  "latitude": 50.0755,
  "longitude": 14.4378,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 17,
  "name": "Brussels",
  "country": "Belgium",
  "population": 1200000,
  "latitude": 50.8503,
  "longitude": 4.3517,
  "is_capital": true,
  "population_category": "Metropolis"
},
{
  "id": 7,
  "name": "Amsterdam",
  "country": "Netherlands",
  "population": 900000,
  "latitude": 52.3676,
  "longitude": 4.9841,
  "is_capital": true,
  "population_category": "Large City"
},
{
  "id": 12,
  "name": "Copenhagen",
  "country": "Denmark",
  "population": 800000,
  "latitude": 55.6761,
  "longitude": 12.5683,
  "is_capital": true,
  "population_category": "Large City"
},

```

```
},
{
  "id": 13,
  "name": "Oslo",
  "country": "Norway",
  "population": 700000,
  "latitude": 59.9139,
  "longitude": 10.7522,
  "is_capital": true,
  "population_category": "Large City"
},
{
  "id": 14,
  "name": "Helsinki",
  "country": "Finland",
  "population": 650000,
  "latitude": 60.1699,
  "longitude": 24.9384,
  "is_capital": true,
  "population_category": "Large City"
},
{
  "id": 18,
  "name": "Zurich",
  "country": "Switzerland",
  "population": 400000,
  "latitude": 47.3769,
  "longitude": 8.5417,
  "is_capital": false,
  "population_category": "City"
}
]
```

○ Performance metrics

```
1762551@L6YT2L4531 hello-map-dublin % curl -o /dev/null -s -w "Response time: %{time_total} seconds\nHTTP status: %{http_code}\n" http://127.0.0.1:8000/api/cities/
Response time: 0.078635 seconds
HTTP status: 200
1762551@L6YT2L4531 hello-map-dublin %
```

4. Report:

- Lab completion summary

In this lab, I developed a Django REST Framework API for managing city data with PostGIS spatial functionality. I implemented the City model with geographic point data and added serializers for different API responses. The API supports standard CRUD operations as well as filtering, GeoJSON output, bounding-box queries and radius-based spatial queries. I also configured Swagger/OpenAPI documentation to make the API endpoints easier to test and understand. Finally, I implemented API tests and carried out manual testing of the main endpoints to verify that the application was working correctly.

- Challenges encountered and solutions

One of the main challenges I encountered was working with Docker Compose and making sure that the Django, PostgreSQL/PostGIS and pgAdmin services were running correctly. I also had to ensure that I was running Docker commands from the correct Week 3 project directory.

- API usage examples

The Cities API provides several endpoints for accessing and querying city data. For example, a GET request to `/api/cities/` can be used to retrieve a list of cities, while `/api/cities/{id}/` can be used to retrieve information about a specific city.

The API also supports spatial queries. For example, the `/api/cities/within-radius/` endpoint accepts a POST request containing a latitude, longitude and radius to find cities within a specified distance.

GeoJSON output can be requested using `/api/cities/geojson/`, which is useful when working with mapping applications. Other endpoints provide information such as available countries and city statistics. These endpoints can be tested directly through the Swagger/OpenAPI interface.

- Future enhancement suggestions

Several future enhancements could be made to the Cities API. The frontend map integration could be developed further to allow users to interact with city data visually rather than accessing it only through the Api. More advanced spatial queries and filtering options could also be introduced to support more specific geographic searches.

Optional Extensions

1. Advanced Features:

- API authentication with token-based access

Token-based authentication was implemented using Django REST Framework's built-in token authentication. A test user and authentication token were created, and the `/api/cities/` endpoint was protected using `IsAuthenticated`.

```
HTTP/1.1 200 OK
Date: Wed, 30 Sep 2026 16:04:32 GMT
Server: WSGIServer/0.2 CPython/3.12.14
Content-Type: application/json
Vary: Accept, Cookie, origin
Allow: GET, POST, HEAD, OPTIONS
X-Frame-Options: DENY
Content-Length: 3247
X-Content-Type-Options: nosniff
Referer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

{"count":20,"next":null,"previous":null,"results":[{"id":3,"name":"Paris","country":"France","population":11000000,"latitude":48.8566,"longitude":2.3522,"is_capital":true,"population_category":"Megacity"}, {"id":2,"name":"London","country":"United Kingdom","population":9000000,"latitude":51.5074,"longitude":0.1278,"is_capital":true,"population_category":"Large Metropolis"}, {"id":5,"name":"Madrid","country":"Spain","population":6600000,"latitude":40.4168,"longitude":3.7038,"is_capital":true,"population_category":"Large Metropolis"}, {"id":8,"name":"Barcelona","country":"Spain","population":5500000,"latitude":41.3851,"longitude":2.1734,"is_capital":false,"population_category":"Large Metropolis"}, {"id":4,"name":"Berlin","country":"Germany","population":3700000,"latitude":52.52,"longitude":13.405,"is_capital":true,"population_category":"Metropolis"}, {"id":20,"name":"Athens","country":"Greece","population":3200000,"latitude":37.9838,"longitude":23.7275,"is_capital":true,"population_category":"Metropolis"}, {"id":6,"name":"Rome","country":"Italy","population":2800000,"latitude":41.9028,"longitude":12.4964,"is_capital":true,"population_category":"Metropolis"}, {"id":19,"name":"Lisbon","country":"Portugal","population":2800000,"latitude":38.7223,"longitude":9.1393,"is_capital":true,"population_category":"Metropolis"}, {"id":10,"name":"Vienna","country":"Austria","population":1900000,"latitude":48.2082,"longitude":16.3738,"is_capital":true,"population_category":"Metropolis"}, {"id":15,"name":"Warsaw","country":"Poland","population":1800000,"latitude":52.2297,"longitude":21.0122,"is_capital":true,"population_category":"Metropolis"}, {"id":11,"name":"Stockholm","country":"Sweden","population":1600000,"latitude":59.3293,"longitude":18.0686,"is_capital":true,"population_category":"Metropolis"}, {"id":9,"name":"Munich","country":"Germany","population":1500000,"latitude":48.1351,"longitude":11.582,"is_capital":false,"population_category":"Metropolis"}, {"id":1,"name":"Dublin","country":"Ireland","population":1400000,"latitude":53.3498,"longitude":6.2603,"is_capital":true,"population_category":"Metropolis"}, {"id":16,"name":"Prague","country":"Czech Republic","population":1300000,"latitude":50.0755,"longitude":14.4378,"is_capital":true,"population_category":"Metropolis"}, {"id":17,"name":"Brussels","country":"Belgium","population":1200000,"latitude":50.8503,"longitude":4.3517,"is_capital":true,"population_category":"Metropolis"}, {"id":7,"name":"Amsterdam","country":"Netherlands","population":900000,"latitude":52.3676,"longitude":4.9041,"is_capital":true,"population_category":"Large City"}, {"id":12,"name":"Copenhagen","country":"Denmark","population":800000,"latitude":55.6761,"longitude":12.5683,"is_capital":true,"population_category":"Large City"}, {"id":13,"name":"Oslo","country":"Norway","population":700000,"latitude":59.9139,"longitude":10.7522,"is_capital":true,"population_category":"Large City"}, {"id":14,"name":"Helsinki","country":"Finland","population":650000,"latitude":60.1699,"longitude":24.9384,"is_capital":true,"population_category":"Large City"}, {"id":18,"name":"Zurich","country":"Switzerland","population":400000,"latitude":47.3769,"longitude":8.5417,"is_capital":false,"population_category":"City"}]}
```

Requests without authentication credentials were rejected with a 403 Forbidden response, while requests containing a valid token successfully returned the city data with a 200 OK response.

```
I762551@L6YT2L453J Week 3 % curl -i http://127.0.0.1:8000/api/cities/
HTTP/1.1 403 Forbidden
Date: Wed, 30 Sep 2026 16:04:01 GMT
Server: WSGIServer/0.2 CPython/3.12.14
Content-Type: application/json
Vary: Accept, Cookie, origin
Allow: GET, POST, HEAD, OPTIONS
X-Frame-Options: DENY
Content-Length: 58
X-Content-Type-Options: nosniff
Referer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

{"detail":"Authentication credentials were not provided."}
```

- Rate limiting for API endpoints

```
HTTP/1.1 429 Too Many Requests
Date: Wed, 30 Sep 2026 16:34:24 GMT
Server: WSGIServer/0.2 CPython/3.12.14
Content-Type: application/json
Retry-After: 38
Vary: Accept, Cookie, origin
Allow: GET, POST, HEAD, OPTIONS
X-Frame-Options: DENY
Content-Length: 69
X-Content-Type-Options: nosniff
Referer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

{"detail":"Request was throttled. Expected available in 38 seconds."}
```

- Caching for improved performance

Django's built-in in-memory caching was configured for the city list endpoint with a cache duration of 60 seconds.

The caching functionality was tested by making two consecutive requests. The first response included `Cache-Control: max-age=60`, while the second response included an `Age` header, demonstrating that the response was being served from the cache.

```

HTTP/1.1 200 OK
Date: Thu, 01 Oct 2026 05:29:48 GMT
Server: WSGIServer/0.2 CPython/3.12.14
Content-Type: application/json
Vary: Accept, Authorization, Cookie, origin
Allow: GET, POST, HEAD, OPTIONS
Expires: Thu, 01 Oct 2026 05:30:48 GMT
Cache-Control: max-age=60
X-Frame-Options: DENY
Content-Length: 3247
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

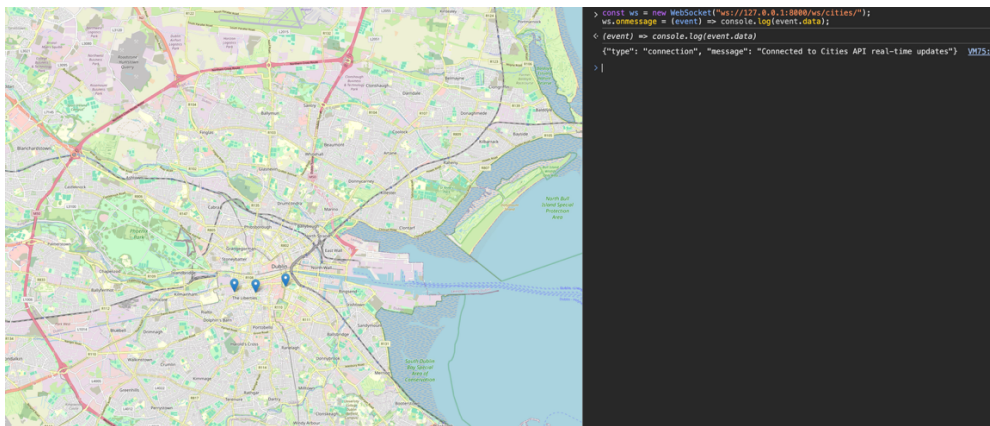
```

```

HTTP/1.1 200 OK
Date: Thu, 01 Oct 2026 05:29:54 GMT
Server: WSGIServer/0.2 CPython/3.12.14
Content-Type: application/json
Vary: Accept, Authorization, Cookie, origin
Allow: GET, POST, HEAD, OPTIONS
Expires: Thu, 01 Oct 2026 05:30:48 GMT
Cache-Control: max-age=60
Age: 6
X-Frame-Options: DENY
Content-Length: 3247
X-Content-Type-Options: nosniff
Referrer-Policy: same-origin
Cross-Origin-Opener-Policy: same-origin

```

- WebSocket integration for real-time updates



2. Data Enhancements:

- Additional city attributes

```

I762551@L6YT2L453J Week 3 % curl http://localhost:8000/api/cities/1/
{"id":1,"name":"Dublin","country":"Ireland","region":"Leinster","population":1400000,"latitude":53.3498,"longitude":-6.2603,"coordinates":[-6.2603,53.3498],"elevation_m":8,"climate":"Oceanic climate","average_temperature_c":12.50,"gdp_eur":26549000000.00,"founded_year":841,"age_years":1185,"is_capital":true,"timezone":"Europe/Dublin","population_category":"Metropolis","created_at":"2026-09-29T16:48:07.119712Z","updated_at":"2026-09-29T16:48:07.119712Z"}

```

- Economic data

```

>>> City.objects.values_list("name", "gdp_eur")
<QuerySet [(('Paris', Decimal('90010000000.00')), ('London', Decimal('53121000000.00')), ('Madrid', Decimal('33717000000.00')), ('Barcelona', Decimal('11221100000.00')), ('Berlin', Decimal('21800000000.00')), ('Athens', Decimal('12124000000.00')), ('Lisbon', Decimal('10000000000.00')), ('Rome', Decimal('21000000000.00')), ('Vienna', Decimal('13150000000.00')), ('Warsaw', Decimal('13274000000.00')), ('Stockholm', Decimal('15133000000.00')), ('Munich', Decimal('12270000000.00')), ('Dublin', Decimal('26549000000.00')), ('Prague', Decimal('24513000000.00')), ('Brussels', Decimal('24779000000.00')), ('Amsterdam', Decimal('20110000000.00')), ('Copenhagen', Decimal('40932000000.00')), ('Oslo', Decimal('21239000000.00')), ('Helsinki', Decimal('10354000000.00')), ('Zurich', Decimal('9239000000.00')))]>

```

- Climate

```

>>> City.objects.values_list("name", "climate", "average_temperature_c")
<QuerySet [(('Paris', 'Oceanic climate', Decimal('14.50')), ('London', 'Oceanic climate', Decimal('13.00')), ('Madrid', 'Semi-arid climate', Decimal('16.50')), ('Barcelona', 'Mediterranean climate', Decimal('18.00')), ('Berlin', 'Oceanic climate', Decimal('12.50')), ('Athens', 'Mediterranean climate', Decimal('19.50')), ('Lisbon', 'Mediterranean climate', Decimal('21.50')), ('Rome', 'Mediterranean climate', Decimal('19.00')), ('Vienna', 'Continental climate', Decimal('12.00')), ('Warsaw', 'Continental climate', Decimal('10.50')), ('Stockholm', 'Continental climate', Decimal('9.50')), ('Munich', 'Continental climate', Decimal('11.50')), ('Dublin', 'Oceanic climate', Decimal('12.50')), ('Prague', 'Continental climate', Decimal('13.50')), ('Brussels', 'Oceanic climate', Decimal('13.50')), ('Amsterdam', 'Maritime climate', Decimal('12.00')), ('Copenhagen', 'Oceanic climate', Decimal('18.00')), ('Oslo', 'Continental climate', Decimal('8.50')), ('Helsinki', 'Continental climate', Decimal('7.50')), ('Zurich', 'Oceanic climate', Decimal('12.00')))]>

```

- Relationships between cities

```

>>> dublin.related_cities.all()
<QuerySet [<City: London, United Kingdom>]>
>>> paris.related_cities.all()
<QuerySet [<City: Brussels, Belgium>]>

```

```

I762551@L6YT2L453J Week 3 % curl http://localhost:8000/api/cities/1/
{"id":1,"name":"Dublin","country":"Ireland","region":"Leinster","population":1400000,"latitude":53.3498,"longitude":-6.2603,"coordinates":[-6.2603,53.3498],"elevation_m":8,"climate":"Oceanic climate","average_temperature_c":12.50,"gdp_eur":26549000000.00,"founded_year":841,"age_years":1185,"is_capital":true,"timezone":"Europe/Dublin","related_cities":[2],"population_category":"Metropolis","created_at":"2026-09-29T16:48:07.119712Z","updated_at":"2026-09-29T16:48:07.119712Z"}

```